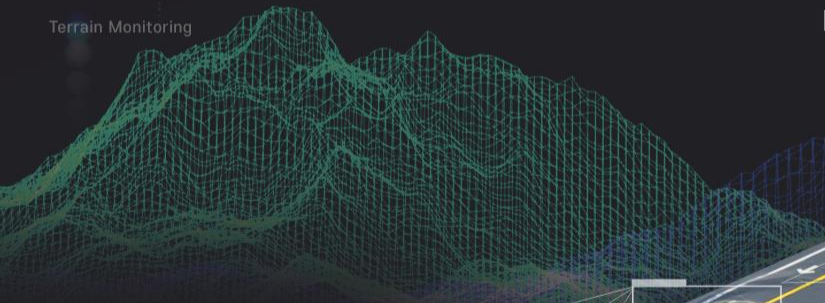


Image Recognition



Terrain Monitoring



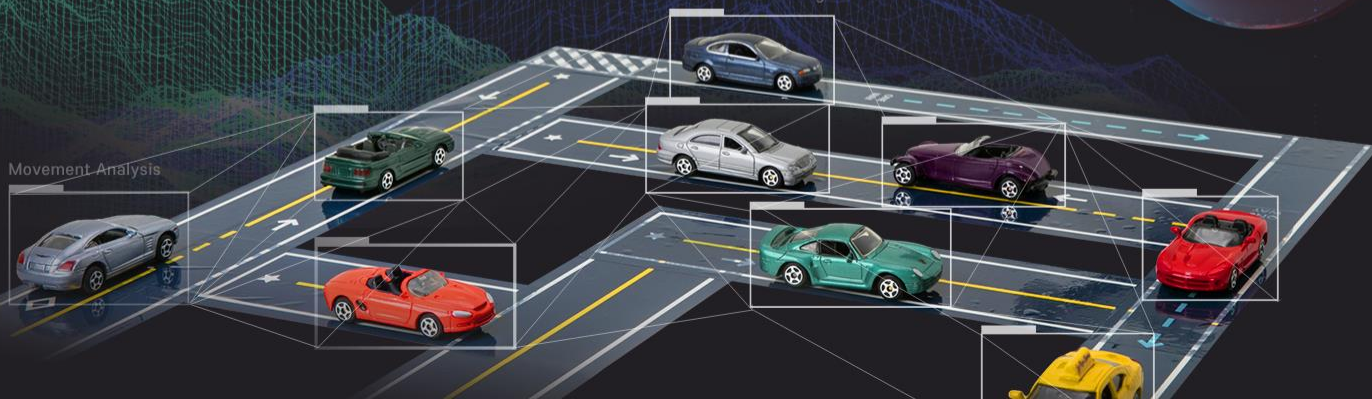
강원혁신플랫폼

컴퓨터 비전

◆
텐서플로의 이해

Autonomous Driving

Movement Analysis



권장

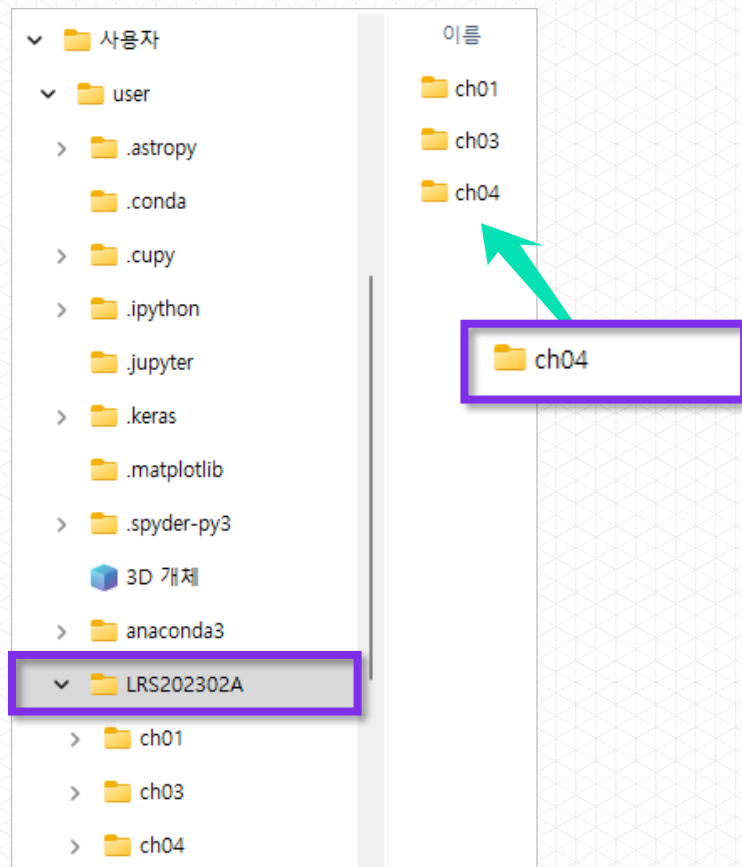
아래와 같은 경로에 실행 소스가 존재하면
환경 구축 완료

c:\Users\user>LRS202302A>

컴퓨터 이름

또는

사용자 계정





■ 딥러닝 소프트웨어의 개발

- ② 딥러닝의 가능성을 인지한 여러 대학과 기업에서는 자체적으로 딥러닝 소프트웨어를 개발해 내부 연구 그룹끼리 공유하기 시작했음
- ② 딥러닝 소프트웨어는 점점 개선되어 큰 규모로 발전했고, 개발자들은 누구나 사용할 수 있도록 라이브러리 형태로 제작해 경쟁적으로 공개함

 TensorFlow	
개발자	구글 브레인팀 ^[1]
발표일	2015년 11월 9일(7년 전)
안정화 버전	2.11.0 ^[2] (2022년 11월 18일(23일 전))
저장소	github.com/tensorflow/tensorflow
프로그래밍 언어	파이썬, C++, CUDA
플랫폼	리눅스, macOS, 마이크로소프트 윈도우, 안드로이드, 자바스크립트
종류	기계 학습 라이브러리
라이선스	아파치 2.0 오픈 소스 라이선스
상태	개발 중
웹사이트	www.tensorflow.org

PyTorch 소프트웨어	<
PyTorch는 Python을 위한 오픈소스 머신 러닝 라이브러리이다. Torch를 기반으로 하며, 자연어 처리와 같은 애플리케이션을 위해 사용된다. GPU사용이 가능하기 때문에 속도가 상당히 빠르다. 위키백과	
최초 출시일: 2016년 9월	
개발자: 페이스북 AI 리서치 랩 (FAIR)	
라이선스: BSD	
안정화 버전: 1.13.1 / 2022년 12월 16일	
프로그래밍 언어: 파이썬, C++	
플랫폼: IA-32, x86-64	
개발: Meta AI, 리눅스 재단	

딥러닝 소프트웨어 자체는 대부분 C와 C++로 개발하지만, 소프트웨어를 불러 쓰는 인터페이스 언어는 파이썬과 같은 스크립트 언어를 채택함



■ 대표적인 딥러닝 소프트웨어

이름	개발 그룹	최초 공개일	작성언어	인터페이스 언어	전이학습 지원	철저한 관리
씨아노(Theano)	몬트리올 대학교	2007년	파이썬	파이썬	O	X
카페(Caffe)	UC버클리	2013년	C++	파이썬, 매트랩, C++	O	X
텐서플로 (TensorFlow)	구글 브레인	2015년	C++, 파이썬, CODA	파이썬, C++, 자바, 자바스크립트, R, Julia, Swift, Go	O	O
케라스(Keras)	프랑소와 솔레	2015년	파이썬	파이썬, R	O	O
파이토치 (PyTorch)	페이스북	2016년	C++, 파이썬, CUDA	파이썬, C++	O	O

- ⊕ 텐서플로는 구글브레인 팀이 2015년에 공개함
- ⊕ 2002년에는 토치(Torch)가 공개되었고, 2016년에는 토치를 파이썬 언어기반으로 바꾼 파이토치(PyTorch)가 페이스북 인공지능 그룹에 의해 공개됨



■ 대표적인 딥러닝 소프트웨어

이름	개발 그룹	최초 공개일	작성언어	인터페이스 언어	전이학습 지원	철저한 관리
씨아노(Theano)	몬트리올 대학교	2007년	파이썬	파이썬	O	X
카페(Caffe)	UC버클리	2013년	C++	파이썬, 매트랩, C++	O	X
텐서플로 (TensorFlow)	구글 브레인	2015년	C++, 파이썬, CODA	파이썬, C++, 자바, 자바스크립트, R, Julia, Swift, Go	O	O
케라스(Keras)	프랑소와 솔레	2015년	파이썬	파이썬, R	O	O
파이토치 (PyTorch)	페이스북	2016년	C++, 파이썬, CUDA	파이썬, C++	O	O

- ⌚ 대학에서 개발한 라이브러리로 몬트리올 대학교에서 개발한 씨아노(Theano)와 캘리포니아 대학교 버클리에서 개발한 카페(Caffe)가 유명함
- ⌚ 현재 텐서플로와 파이토치가 높은 점유율을 차지하고 있으며, 텐서플로는 기업에서 선호하고 파이토치는 대학 연구자들이 많이 사용함

텐서플로(TensorFlow)

강력한 수치 계산을 라이브러리

 TensorFlow	
개발자	구글 브레인 팀 ^[1]
발표일	2015년 11월 9일(7년 전)
안정화 버전	2.11.0 ^[2] (2022년 11월 18일(23일 전))
저장소	github.com/tensorflow/tensorflow
프로그래밍 언어	파이썬, C++, CUDA
플랫폼	리눅스, macOS, 마이크로소프트 윈도우, 안드로이드, 자바스크립트
종류	기계 학습 라이브러리
라이선스	아파치 2.0 오픈 소스 라이선스
상태	개발 중
웹사이트	www.tensorflow.org

대규모 머신러닝에 잘 맞도록 튜닝되어 있음
하지만, **계산량이 많이 필요한 어떤 작업에도 사용할 수 있음**

텐서플로(TensorFlow)

강력한 수치 계산을 라이브러리

머신러닝과 딥러닝을 위한 오픈소스 플랫폼

구글의 인공지능 개발부서에서 개발하여 사용하다가
2015년에 오픈소스로 공개되어 현재 텐서플로 2 버전으로 발전함

머신러닝과 딥러닝을 위한 플랫폼은 텐서플로, 테노아(Theano),
파이토치(Pytorch), Mxnet, 사이킷런(Scikit-learn), NLTK 등 여러 종류가 있음

텐서플로(TensorFlow)

강력한 수치 계산을 라이브러리

- ☞ 현재 가장 많은 사용자를 확보하고 있는 좋은 플랫폼
- ☞ 우수한 기능과 서비스를 제공하고 있으며, 병렬처리를 잘 지원하며 고급 신경망 네트워크 모델을 쉽게 구현할 수 있어 인기 있음
- ☞ 2019년 10월 2.0 버전을 발표하면서 케라스(Keras)가 텐서플로에 편입됨
- ☞ 버전 2.0은 Tensorflow와 Numpy가 호환됨



■ 텐서플로의 주요 기능

- 1 ➤ 핵심 구조는 넘파이와 매우 비슷하지만 GPU를 지원함
- 2 ➤ (여러 장치와 서버에 대해서) 분산 컴퓨팅을 지원함
- 3 ➤ 일종의 JIT(Just-In-Time) 컴파일러를 포함함
 - ⌚ 속도를 높이고 메모리 사용량을 줄이기 위해 계산을 최적화함
 - ⌚ 이를 위해 파이썬 함수에서 계산 그래프(Computation graph)를 추출한 다음 최적화(사용하지 않는 노드를 가지치기)하고 효율적으로 실행함(독립적인 연산을 자동으로 병렬 실행)
- 4 ➤ 자동 미분(Autodiff) 기능과 RMSProp, Nadam 같은 고성능 옵티마이저를 제공하므로 모든 종류의 손실 함수를 쉽게 최소화할 수 있음



■ 텐서플로의 전체 파이썬 API 요약

TensorFlow's python API

tf.keras tf.estimator	High-level Deep Learning APIs	tf.distribute tf.saved_model tf.autograph tf.graph_util tf.lite tf.quantization tf.tpu tf.xla	Deployment and optimization
tf.nn tf.losses tf.metrics tf.optimizers tf.train tf.initializers	Low-level Deep Learning APIs		
tf.GradientTape tf.gradients ()	Autodiff	tf.lookup tf.nest tf.ragged tf.sets tf.sparse tf.strings	Special data structures
tf.data tf.feature_column tf.audio tf.image tf.io tf.queue	I/O and preprocessing	tf.math tf.linalg tf.signal tf.random tf.bitwise	Mathematics, including linear algebra and signal processing
tf.summary	Visualization with TensorBoard	tf.compat tf.config & more	Miscellaneous

영문명칭	한글명칭
High-level Deep Learning APIs	고수준 딥러닝 API
Low-level Deep Learning APIs	저수준 딥러닝 API
Autodiff	자동 미분
I/O and preprocessing	입출력과 전처리
Visualization with TensorBoard	텐서보드 시각화
Deployment and optimization	배포와 최적화
Special data structures	특수한 데이터 구조
Mathematics, including linear algebra and signal processing	선형대수와 신호처리를 포함한 수학연산
Miscellaneous	그 외



■ 가장 저수준의 텐서플로 연산(Operation, 줄여서 op)은 매우 효율적인 C++ 코드로 구현되어 있음

- ↻ 많은 연산은 커널(Kernel)이라 부르는 여러 구현을 가짐
- ↻ 각 커널은 CPU, GPU, 또는 TPU(텐서 처리 장치, Tensor processing unit)와 같은 특정 장치에 맞추어 만들어짐

GPU

계산을 작은 단위로 나누어
여러 GPU 스레드에서
병렬로 실행하므로
속도를 극적으로 향상함

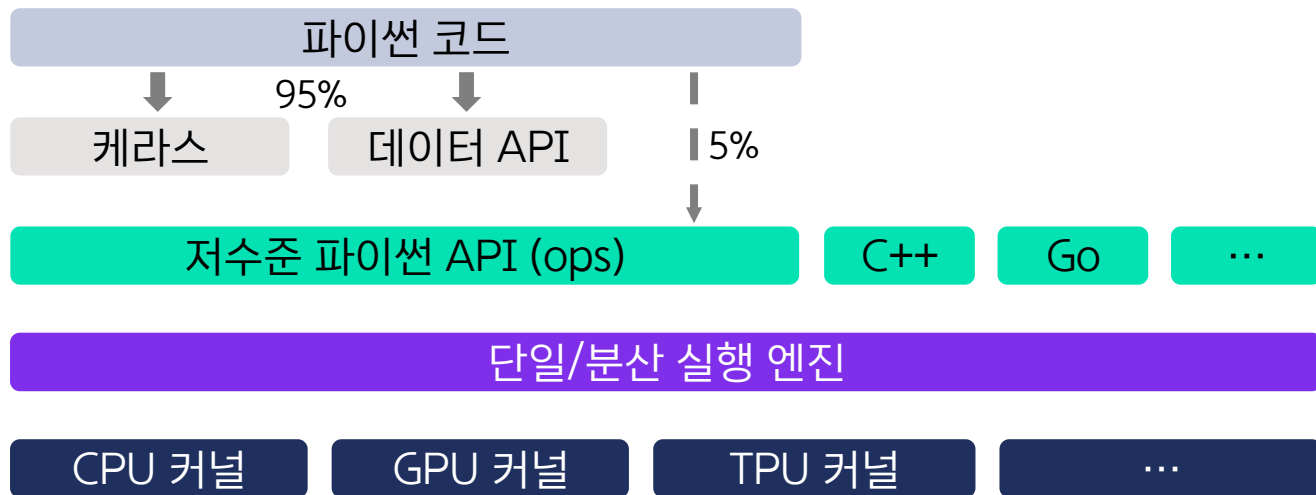
TPU

딥러닝 연산을 위해
특별하게 설계된 ASIC 칩

■ 텐서플로 구조

- ➡ 대부분의 코드는 **고수준 API**를 사용함
- ➡ 더 높은 자유도가 필요한 경우 **저수준 파이썬 API**를 사용하여 **텐서를 직접 다루게 됨**

텐서플로 구조





■ 텐서플로의 실행

- ② 윈도우, 리눅스, 맥OS뿐만 아니라(텐서플로 Lite를 사용하여) iOS와 안드로이드 같은 모바일 장치에도 실행됨
- ② 파이썬 API 외에도 C++, 자바, Go, 스위프트 API를 사용할 수 있음
- ② TensorFlow.js라는 자바스크립트 구현도 있으며, 이를 사용하면 브라우저에서 직접 모델을 실행하는 것이 가능함



텐서플로 라이브러리의 버전을 확인해 보자.

- 주피터 노트북에서 텐서플로 버전을 확인
- 텐서플로(=2.9.2) tensorflow 라이브러리를 설치
- 설치하는 경우 !pip3 install tensorflow==2.9.2 명령어를 사용해 설치한 후에 확인

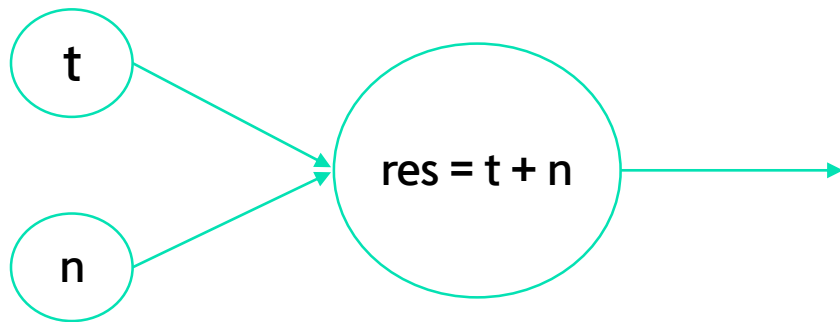
```
import tensorflow as tf

print(tf.__version__)          # tensorflow 버전
a = tf.random.uniform([2,3],0,1)
print(a)
print(type(a))
```

```
2.9.2
tf.Tensor(
[[0.16749144 0.34703743 0.980523  ]
 [0.6586274  0.8499924  0.6469109 ]], shape=(2, 3), dtype=float32)
<class 'tensorflow.python.framework.ops.EagerTensor'>
```

텐서플로 자료구조를 이용해 생성한 행렬 t 와
`numpy`로 생성한 행렬 n 의 호환성을 확인해 보자.

- ➡ 텐서플로 버전 2.0부터는 `tensorflow`와 `numpy`가 호환됨
- ➡ `tensorflow`와 `numpy`가 호환되는지 파이썬 코드로 확인



계산 절차를 표현한 계산 그래프



다음은 tensorflow와 numpy가 호환되는지 확인하는 파이썬 코드이다.

실행 결과 아무 문제없이 덧셈이 이루어지고 결과가 출력된 것을 볼 수 있음

```
import tensorflow as tf
import numpy as np
```

```
t = tf.random.uniform([2,3],0,1)
n = np.random.uniform(0,1,[2,3])
print("tensorflow로 생성한 텐서:\n",t,"\n")
print("numpy로 생성한 ndarray:\n",n,"\n")
```

```
res = t + n # 텐서 t와 ndarray n의 덧셈
print("덧셈 결과:\n",res)
```

tensorflow로 생성한 텐서:

```
tf.Tensor(
[[[0.49070978 0.12943375 0.34216154]
 [0.67453706 0.04041588 0.20851016]]], shape=(2, 3), dtype=float32)
```

numpy로 생성한 ndarray:

```
[[[0.02061141 0.3516221 0.64222089]
 [0.07061457 0.98339623 0.62322629]]]
```

덧셈 결과:

```
tf.Tensor(
[[[0.5113212 0.48105586 0.98438245]
 [0.74515164 1.023812 0.83173645]]], shape=(2, 3), dtype=float32)
```



■ 텐서(Tensor)란?

- 기계학습에서는 데이터가 가장 중요한데, 딥러닝에서는 데이터를 텐서(Tensor)라는 자료구조로 표현함
- 신경망의 가중치도 텐서로 표현함
- 딥러닝에서는 아래 그림과 같은 다차원 배열(Muti-dimensional array)을 텐서라고 함



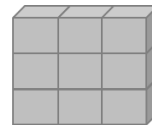
3.2

(a) 0차원 텐서(스칼라)

(3.2, -1.2, 0.01)

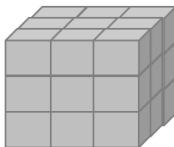


(b) 1차원 텐서(벡터)

$$\begin{pmatrix} 1.2 & -0.1 & 0.1 \\ 2.1 & 0.2 & 2.4 \\ 4.5 & -1.0 & 2.3 \end{pmatrix}$$


(c) 2차원 텐서(행렬)

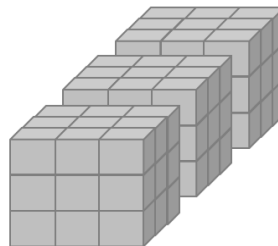
텐서의 구조

$$\begin{pmatrix} 1.2 & -0.1 & 0.1 & 2.4 \\ 2.1 & 0.2 & 2.4 & 2.3 \\ 4.5 & -1.0 & 2.3 & 2.3 \end{pmatrix}$$


(d) 3차원 텐서

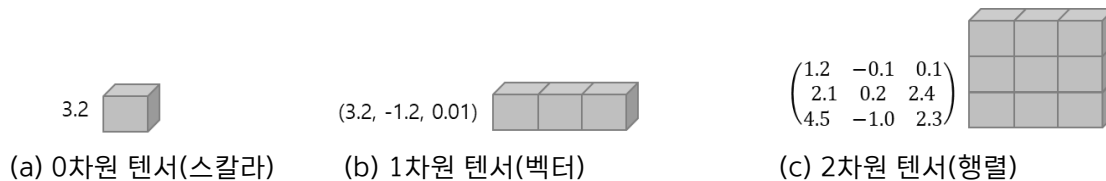
$$\begin{pmatrix} 1.2 & -0.1 & 0.1 & 2.4 \\ 2.1 & 0.2 & 2.4 & 2.3 \\ 4.5 & -1.0 & 2.3 & 2.3 \end{pmatrix}$$

(e) 4차원 텐서

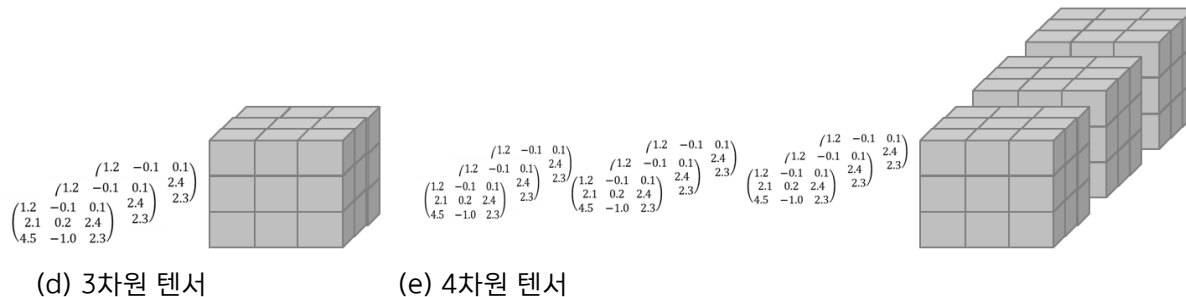


■ 텐서(Tensor)란?

- 프로그래머는 자신이 다루는 데이터의 특성을 정확히 이해해야만 데이터를 텐서플로에 입력하고 학습할 수 있음
- 데이터의 특성 중에서 가장 중요한 것은 텐서의 구조임
- 아래의 그림은 0차원, 1차원, 2차원, 3차원, 4차원 구조의 텐서를 보여줌



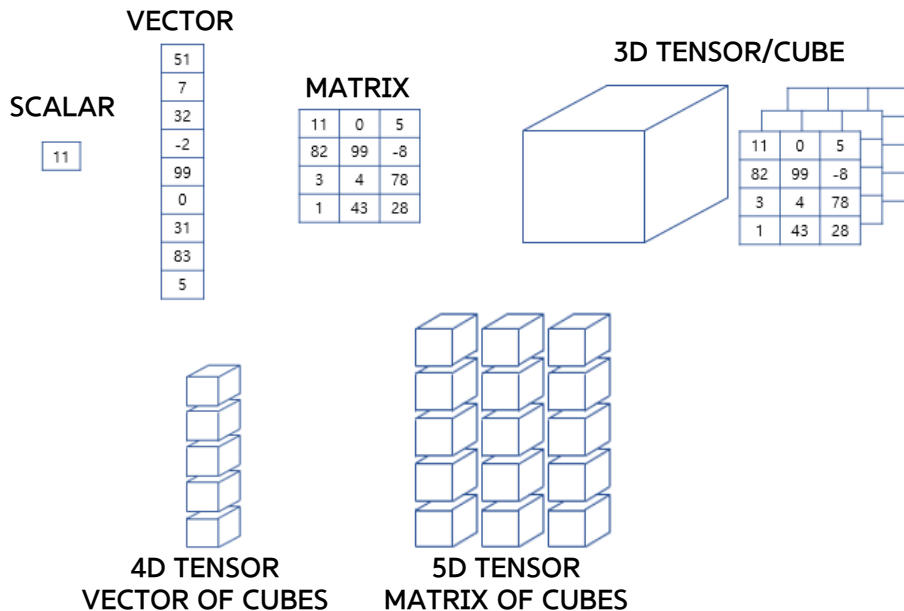
텐서의 구조





■ 텐서(Tensor)란?

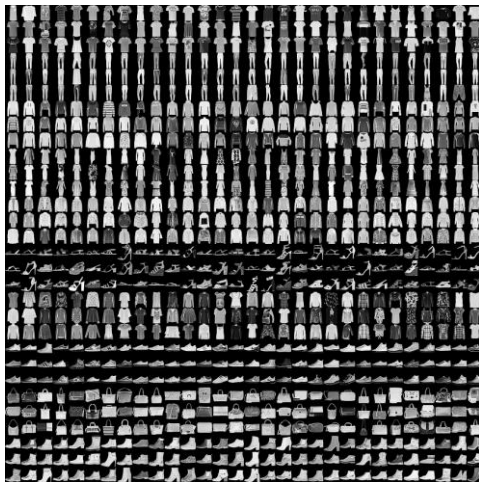
- 동영상 여러 개로 구성된 데이터셋은 5차원 텐서임
- 이론상 텐서플로는 무한 차원의 텐서를 다룰 수 있는데 보통 5차원 정도까지 다룸



텐서플로가 제공하는 **MNIST**(손글씨 이미지, 0~9 10개의 클래스, 28×28, 60000개 흑백 이미지), **Fashion-MNIST**(각 이미지는 10개의 클래스, 28×28, 70,000개 이미지), **Boston Housing**(보스턴 시의 주택 가격 데이터), **Reuters**(로이터 뉴스 데이터) 네 가지 데이터 셋을 읽고 **텐서의 구조를 확인**해 보자.

Fashion-MNIST 데이터셋은 **28×28 픽셀의 70,000개** 이미지,
각 이미지는 10개의 클래스로 라벨링이 되어 있음

- 0 : T-shirt/top
- 1 : Trouser
- 2 : Pullover
- 3 : Dress
- 4 : Coat
- 5 : Sandal
- 6 : Shirt
- 7 : Sneaker
- 8 : Bag
- 9 : Ankle boot



텐서플로가 제공하는 **MNIST**(손글씨 이미지, 0~9 10개의 클래스, 28×28, 60000개 흑백 이미지), **Fashion-MNIST**(각 이미지는 10개의 클래스, 28×28, 70,000개 이미지), **Boston Housing**(보스턴 시의 주택 가격 데이터), **Reuters**(로이터 뉴스 데이터) 네 가지 데이터 셋을 읽고 텐서의 구조를 확인해 보자.

CIFAR-10 데이터셋(Canadian Institute For Advanced Research)은 **32×32 픽셀의 60,000개** 컬러 이미지, 각 이미지는 10개의 클래스로 라벨링이 되어 있음

비행기

자동차

새

고양이

사슴

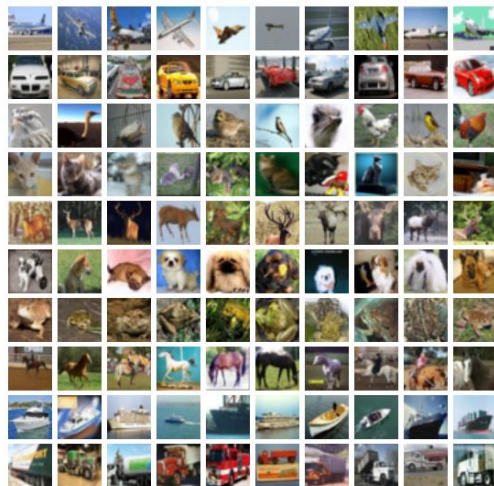
개

개구리

말

배

트럭



50,000개 이미지는 훈련 데이터,
10,000개 이미지는 테스트 데이터

다음은 텐서플로가 제공하는 MNIST, Fashion-MNIST, Boston Housing, Reuters 네 가지 데이터 셋을 읽고 텐서의 구조를 확인하는 파이썬 코드이다.

다음은 텐서플로가 제공하는 MNIST 데이터 셋을 읽고 텐서의 구조를 확인하는 코드이다.

```
import tensorflow as tf
import tensorflow.keras.datasets as ds

# MNIST 읽고 텐서 모양 출력
(x_train, y_train), (x_test, y_test) = ds.mnist.load_data()
yy_train = tf.one_hot(y_train, 10, dtype=tf.int8) # 원핫 코드로 변환
print("MNIST: ", x_train.shape, y_train.shape, yy_train.shape)
# MNIST: (60000, 28, 28) (60000,) (60000, 10)
```

- ➡ MNIST 데이터셋의 훈련 데이터는 3차원 텐서 구조를 가지며 크기가 60000×28×28임
- ➡ 레이블 정보는 1차원 텐서 구조이며 크기가 60000임을 알 수 있음
- ➡ 원핫 코드 변환은 2차원 텐서이며 크기가 60000×10임

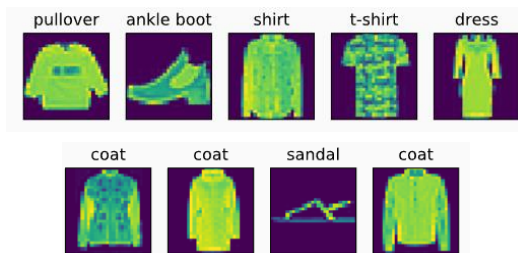
⊕ 텐서플로가 제공하는 데이터셋의 텐서 구조

다음은 텐서플로가 제공하는 Fashion-MNIST 데이터 셋을 읽고
텐서의 구조를 확인하는 파이썬 코드이다.

```
(x_train,y_train),(x_test,y_test) = ds.fashion_mnist.load_data()  
yy_train = tf.one_hot(y_train,10,dtype=tf.int8)  
print("Fashion-MNIST: ",x_train.shape,y_train.shape,yy_train.shape)  
# Fashion-MNIST:(60000, 28, 28) (60000,) (60000, 10)
```

Fashion-MNIST 데이터셋은 10개의 클래스로 나뉘져 있으며 각 번호는 다음과 같음

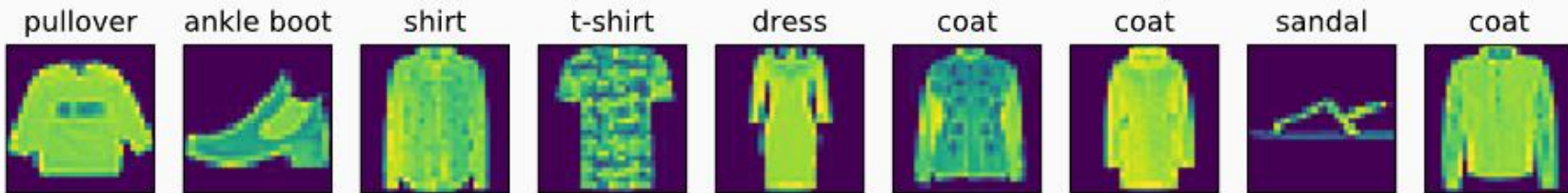
- 0 : T-shirt/top(티셔츠/탑)
- 1 : Trouser(바지)
- 2 : Pullover(풀오버-스웨터의 일종)
- 3 : Dress(드레스)
- 4 : Coat(코트)
- 5 : Sandal(샌들)
- 6 : Shirt(셔츠)
- 7 : Sneaker(스니커즈)
- 8 : Bag(가방)
- 9 : Ankle boot(앵클 부츠)



다음은 텐서플로가 제공하는 Fashion-MNIST 데이터 셋을 읽고
텐서의 구조를 확인하는 파이썬 코드이다.

Fashion-MNIST 데이터셋은 10개의 클래스로 나뉘져 있으며 각 번호는 다음과 같음

- 0 : T-shirt/top(티셔츠/탑)
- 1 : Trouser(바지)
- 2 : Pullover(풀오버-스웨터의 일종)
- 3 : Dress(드레스)
- 4 : Coat(코트)
- 5 : Sandal(샌들)
- 6 : Shirt(셔츠)
- 7 : Sneaker(스니커즈)
- 8 : Bag(가방)
- 9 : Ankle boot(앵클 부츠)



다음은 텐서플로가 제공하는 **Boston Housing** 데이터 셋을 읽고
텐서의 구조를 확인하는 파이썬 코드이다.

```
# Boston Housing 읽고 텐서 모양 출력
(x_train,y_train),(x_test,y_test) = ds.boston_housing.load_data()
print("Boston Housing: ",x_train.shape,y_train.shape)
# Boston Housing: (404, 13) (404,)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/boston\_housing.npz
57026/57026 [=====] - 0s 0us/step
Boston Housing:  (404, 13) (404,)
```

보스턴 시의 주택 가격 데이터(Boston Housing)는 404채의 주택에 대한 방의 수,
동네 범죄율, 고속도로까지 거리 등의 13개 속성을 기록하고 있음

다음은 텐서플로가 제공하는 Reuters 데이터 셋을 읽고
텐서의 구조를 확인하는 파이썬 코드이다.

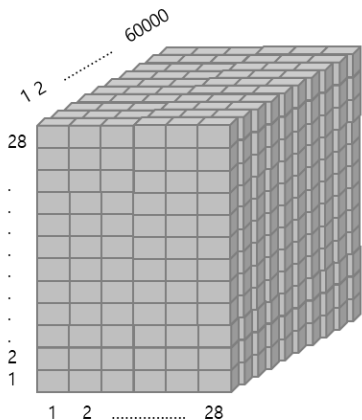
```
# Reuters 읽고 텐서 모양 출력
(x_train,y_train),(x_test,y_test) = ds.reuters.load_data()
print("Reuters: ",x_train.shape,y_train.shape) # Reuters: (8982,) (8982,)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/reuters.npz
2110848/2110848 [=====] - 0s 0us/step
Reuters:  (8982,) (8982,)
```

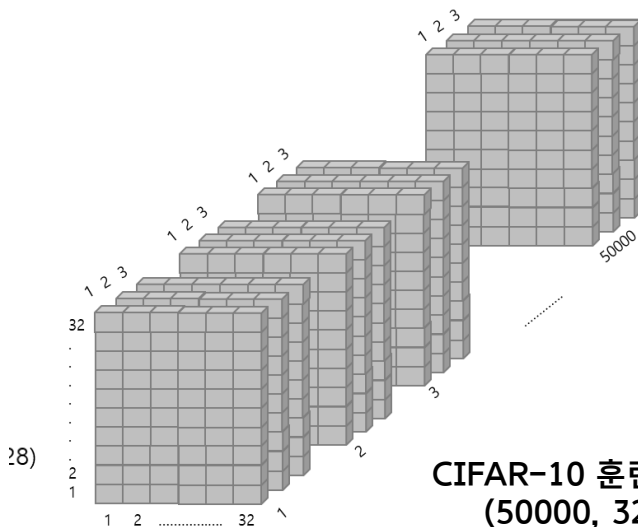
로이터 뉴스 데이터(Reuters)는
텍스트를 저장한 (8992,) 모양의 1차원 텐서 구조임

아래 그림과 같이 MNIST 데이터셋은 3차원 텐서 구조를 가지며 크기가 $60000 \times 28 \times 28$ 이고,
CIFAR-10 데이터셋은 4차원 텐서 구조를 가지며 크기가 $50000 \times 32 \times 32 \times 3$ 이다.

MNIST의 한 장의 이미지는 28×28 픽셀의 흑백 이미지이고,
CIFAR-10의 한 장의 이미지는 32×32 크기의 R맵, G맵, B맵으로 구성된 컬러 이미지임



MNIST 훈련데이터 텐서
(60000, 28, 28)



CIFAR-10 훈련데이터 텐서
(50000, 32, 32, 3)